

DISEÑO DE PATRONES – SEMANA 08

Ejercicios Prácticos - Patrón Estructural Adapter y Facade

Ejercicio: Una empresa cuenta con un sistema antiguo que registra la información de sus clientes en un formato plano de texto, pero la nueva aplicación de pedidos requiere los datos en un formato estructurado (JSON); para integrar ambos sistemas sin modificar el código legado se implementa el patrón **Adapter**, donde el sistema antiguo actúa como *Adaptee*, la interfaz define el *Target*, el adaptador transforma los datos al formato requerido y el cliente demuestra cómo la aplicación antigua y la nueva reciben la información en diferentes representaciones.

Proyecto: Adapter/

```
└─ Package: Clientepedido/
    ├── SistemaClientesAntiguo.java (Adaptee)
    ├── ClienteTarget.java (Target)
    ├── ClienteAdapter.java (Adapter)
    └── ClienteDemo.java (Client - main)
```

Pestaña SistemaClientesAntiguo.java

```
package Clientepedido;

/**
 * Sistema antiguo que devuelve datos de clientes en formato simple.
 */
public class SistemaClientesAntiguo {
    public String obtenerCliente() {
        return "Cliente: Juan Pérez, Dirección: Lima, Teléfono: 999999999";
    }
}
```

Pestaña ClienteTarget.java

```
package Clientepedido;
public interface ClienteTarget {
    String getDatosCliente();
}
```

Pestaña ClienteAdapter.java

```
package Clientepedido;
public class ClienteAdapter implements ClienteTarget {
    private SistemaClientesAntiguo sistemaAntiguo;

    public ClienteAdapter(SistemaClientesAntiguo sistemaAntiguo) {
        this.sistemaAntiguo = sistemaAntiguo;
    }
    @Override
    public String getDatosCliente() {
        String datosAntiguos = sistemaAntiguo.obtenerCliente();
        // Transformación simulada a formato JSON
        String datosNuevos = "{ \"nombre\": \"Juan Pérez\", \"direccion\": \"Lima\", \"telefono\": \"999999999\" }";
        return datosNuevos;
    }
}
```

DISEÑO DE PATRONES – SEMANA 08

Ejercicios Prácticos - Patrón Estructural Adapter y Facade

Pestaña ClienteDemo.java

```
}  
}  
  
package Clientepedido;  
public class ClienteDemo {  
    public static void main(String[] args) {  
        SistemaClientesAntiguo sistemaAntiguo = new SistemaClientesAntiguo();  
        ClienteTarget adaptador = new ClienteAdapter(sistemaAntiguo);  
  
        System.out.println("=== EJECUCIÓN DEL PROGRAMA ===");  
        System.out.println("Datos recibidos por la aplicación ANTIGUA:");  
        System.out.println(sistemaAntiguo.obtenerCliente());  
  
        System.out.println("\nDatos recibidos por la aplicación NUEVA:");  
        System.out.println(adaptador.getDatosCliente());  
    }  
}
```

=====

Proyecto: Hospital

|
Package: Facade
|----> SistemaEMR
|----> SistemaLaboratorio
|----> SistemaFacturacion
|----> SistemaFacadeHospital
|----> Main

Pestaña SistemaEMR.java

```
package com.utp.hospitalfacade;  
public class SistemaEMR {  
    public String obtenerHistoriaClinica(String paciente) {  
        return "Historia clínica de " + paciente + ": [diagnóstico, tratamientos previos]";  
    }  
}
```

Pestaña SistemaLaboratorio.java

```
package com.utp.hospitalfacade;  
public class SistemaLaboratorio {  
    public String obtenerResultados(String paciente) {  
        return "Resultados de laboratorio de " + paciente + ": [hemograma, glucosa, colesterol]";  
    }  
}
```

Pestaña SistemaLaboratorio.java

```
package com.utp.hospitalfacade;  
public class SistemaFacturacion {  
    public String obtenerEstadoCuenta(String paciente) {  
        return "Estado de facturación de " + paciente + ": [saldo pendiente, pagos realizados]";  
    }  
}
```

DISEÑO DE PATRONES – SEMANA 08

Ejercicios Prácticos - Patrón Estructural Adapter y Facade

Pestaña FacadeHospital.java

```
package com.utp.hospitalfacade;  
public class FacadeHospital {  
    private SistemaEMR emr;  
    private SistemaLaboratorio laboratorio;  
    private SistemaFacturacion facturacion;  
    public FacadeHospital() {  
        emr = new SistemaEMR();  
        laboratorio = new SistemaLaboratorio();  
        facturacion = new SistemaFacturacion();  
    }  
    public void mostrarInformacionCompleta(String paciente) {  
        System.out.println("=== Información completa del paciente ===");  
        System.out.println(emr.obtenerHistoriaClinica(paciente));  
        System.out.println(laboratorio.obtenerResultados(paciente));  
        System.out.println(facturacion.obtenerEstadoCuenta(paciente));  
        System.out.println("=====");  
    }  
}
```

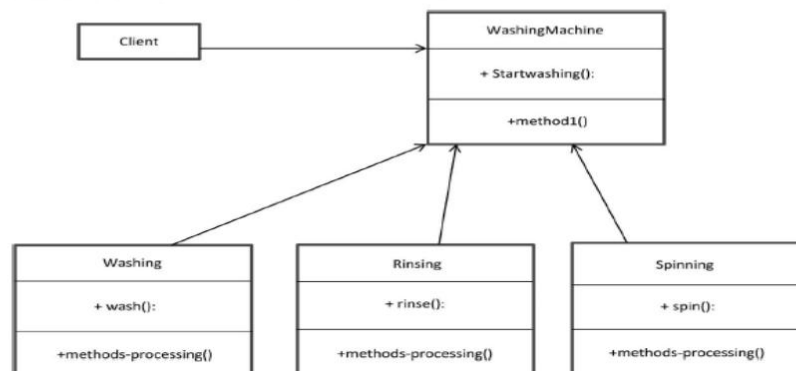
Pestaña Main.java

```
package com.utp.hospitalfacade;  
public class Main {  
    public static void main(String[] args) {  
        FacadeHospital fachada = new FacadeHospital();  
        fachada.mostrarInformacionCompleta("Juan Pérez");  
    }  
}
```

Patrones estructural – Facade - ejemplo



Supongamos que tenemos una lavadora que puede lavar, enjuagar y centrifugar la ropa, pero que realiza cada tarea por separado. Debemos abstraer las complejidades de los subsistemas porque el sistema en su conjunto es bastante complejo. Necesitamos un sistema que pueda automatizar toda la tarea sin nuestra interferencia.



DISEÑO DE PATRONES – SEMANA 08

Ejercicios Prácticos - Patrón Estructural Adapter y Facade

Proyecto : LAVADORA/

└─ Package : FACADE/

└─ Lavado.java

└─ Enjuague.java

└─ Centrifugado.java

└─ FacadeLavadora.java

└─ Principal.java

Pestaña Lavado.java

```
package FACADE;
```

```
/**
 * Subsistema de lavado.
 * Encargado de realizar la acción de lavar la ropa.
 */
public class Lavado {
    public void lavar() {
        System.out.println("Lavando la ropa con detergente...");
    }
}
```

Pestaña Enjuague.java

```
package FACADE;
```

```
/**
 * Subsistema de enjuague.
 * Encargado de enjuagar la ropa con agua limpia.
 */
public class Enjuague {
    public void enjuagar() {
        System.out.println("Enjuagando la ropa con agua limpia...");
    }
}
```

Pestaña Centrifugado.java

```
package FACADE;
```

```
/**
 * Subsistema de centrifugado.
 * Encargado de quitar el exceso de agua de la ropa.
 */
public class Centrifugado {
    public void centrifugar() {
        System.out.println("Centrifugando la ropa para quitar el exceso de agua...");
    }
}
```

DISEÑO DE PATRONES – SEMANA 08

Ejercicios Prácticos - Patrón Estructural Adapter y Facade

Pestaña FacadeLavadora.java

```
package FACADE;
```

```
/**
 * Clase Facade que simplifica el uso de la lavadora.
 * Permite al cliente ejecutar un ciclo completo sin preocuparse
 * por los detalles internos de cada subsistema.
 */
public class FacadeLavadora {
    private Lavado lavado;
    private Enjuague enjuague;
    private Centrifugado centrifugado;

    public FacadeLavadora() {
        lavado = new Lavado();
        enjuague = new Enjuague();
        centrifugado = new Centrifugado();
    }

    /**
     * Método que automatiza todo el proceso de lavado.
     */
    public void iniciarLavadoCompleto() {
        System.out.println("=== Iniciando ciclo completo de lavado ===");
        lavado.lavar();
        enjuague.enjuagar();
        centrifugado.centrifugar();
        System.out.println("=== Ciclo de lavado terminado ===");
    }
}
```

Pestaña Principal.java

```
package FACADE;
```

```
/**
 * Clase principal (Cliente).
 * Demuestra el uso del patrón Facade en una lavadora.
 */
public class Principal {
    public static void main(String[] args) {
        FacadeLavadora lavadora = new FacadeLavadora();
        lavadora.iniciarLavadoCompleto();
    }
}
```